HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN MÔN THỰC TẬP CƠ SỞ

Đề tài: Xây dựng hệ thống chatbot pháp luật ứng dụng RAG

Nhóm 2

Trần Mạnh Dũng	B23DCCN215
Đỗ Minh Vũ	B23DCCN942

Hà Nội - 2026

MỤC LỤC

PHẦN 1. ĐẶT VẤN ĐỀ 3

1.1. Bối cảnh và lý do chọn đề tài

1.2. Mục tiêu của hệ thống

1.3. Phạm vi thực hiện

1.4. Cơ sở lý thuyết

PHẦN 2. THIẾT KẾ 6

2.1. Yêu cầu hệ thống

2.2. Kiến trúc tổng thể

2.3. Thiết kế dữ liệu

2.4. Thiết kế luồng xử lý RAG

2.5. Thiết kế giao diện và API

PHẦN 3. CÀI ĐẶT 10

3.1. Môi trường và công nghệ sử dụng

3.2. Cấu trúc thư mục dự án

3.3. Cài đặt hạ tầng

3.4. Cài đặt backend

3.5. Cài đặt frontend

3.6. Cài đặt các thành phần RAG

PHẦN 4. KIỂM THỬ 14

4.1. Mục tiêu kiểm thử

4.2. Kịch bản kiểm thử

4.3. Đánh giá kết quả

PHẦN 5. KẾT LUẬN 16

5.1. Kết quả đạt được

5.2. Hạn chế

5.3. Hướng phát triển

PHẦN 1. ĐẶT VẤN ĐỀ

1.1. Bối cảnh và lý do chọn đề tài

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ, nhu cầu tra cứu thông tin pháp luật một cách nhanh chóng, chính xác và dễ hiểu ngày càng trở nên cần thiết. Các văn bản pháp luật thường có dung lượng lớn, cấu trúc phức tạp, nhiều điều khoản, nhiều phụ lục và thường xuyên được cập nhật. Điều này khiến người dùng phổ thông gặp khó khăn khi cần tìm kiếm hoặc hiểu một quy định cụ thể.

Đối với sinh viên, doanh nghiệp hoặc người dân, việc đọc thủ công toàn bộ văn bản pháp luật mất nhiều thời gian và dễ dẫn đến hiểu sai nếu thiếu kiến thức nền tảng. Trong khi đó, các công cụ tìm kiếm truyền thống chủ yếu trả về danh sách tài liệu, chưa trực tiếp giải thích nội dung theo ngôn ngữ tự nhiên.

Sự phát triển của trí tuệ nhân tạo, đặc biệt là các mô hình ngôn ngữ lớn, đã mở ra hướng tiếp cận mới trong việc xây dựng các hệ thống hỏi đáp thông minh. Tuy nhiên, nếu chỉ sử dụng mô hình ngôn ngữ lớn đơn thuần, hệ thống có thể sinh ra thông tin không chính xác hoặc không có căn cứ rõ ràng. Với lĩnh vực pháp luật, vấn đề này đặc biệt nghiêm trọng vì câu trả lời cần bám sát nguồn tài liệu đáng tin cậy.

Từ những lý do trên, nhóm lựa chọn đề tài "Xây dựng hệ thống Chatbot pháp luật ứng dụng RAG". Hệ thống kết hợp giữa truy xuất tài liệu và sinh câu trả lời bằng mô hình ngôn ngữ lớn. Thay vì trả lời hoàn toàn dựa trên kiến thức đã học của mô hình, chatbot sẽ tìm các đoạn văn bản liên quan trong kho tài liệu, sau đó dùng các đoạn này làm ngữ cảnh để sinh câu trả lời.

1.2. Mục tiêu của hệ thống

Mục tiêu tổng quát của đề tài là xây dựng một hệ thống chatbot có khả năng hỗ trợ người dùng hỏi đáp thông tin pháp luật dựa trên tài liệu đã được cung cấp. Hệ thống hướng đến việc tăng độ chính xác, hạn chế hiện tượng sinh thông tin sai và giúp người dùng tiếp cận văn bản pháp luật một cách thuận tiện hơn.

- Xây dựng một ứng dụng web full-stack gồm backend FastAPI và frontend React + Vite.
- Cho phép người dùng đăng ký, đăng nhập và quản lý phiên làm việc.
- Cho phép người dùng tải lên tài liệu pháp luật ở định dạng PDF, TXT hoặc Markdown.
- Tự động trích xuất nội dung từ tài liệu, bao gồm cả PDF có sẵn text và PDF dạng scan.

- Áp dụng OCR offline bằng PyMuPDF và RapidOCR để xử lý tài liệu scan khi cần thiết.
- Chia nhỏ văn bản theo header nhằm giữ cấu trúc chương, mục, điều, khoản của tài liệu pháp luật.
- Lưu vector embedding và metadata của chunk vào Qdrant để phục vụ truy xuất ngữ nghĩa.
- Kết hợp truy xuất vector, truy xuất từ khóa và rerank để chọn ra ngữ cảnh phù hợp nhất.
- Phân rã câu hỏi của người dùng thành các truy vấn con để tăng khả năng bao phủ thông tin.
- Sử dụng Tavily làm web fallback khi tài liệu nội bộ không đủ mạnh để trả lời.
- Sử dụng Gemini để sinh câu trả lời cuối cùng dựa trên ngữ cảnh đã truy xuất.
- Lưu thông tin người dùng, tài liệu tải lên và lịch sử hội thoại bằng MongoDB.

1.3. Phạm vi thực hiện

Phạm vi của đề tài tập trung vào xây dựng hệ thống chatbot pháp luật ở mức ứng dụng thử nghiệm. Hệ thống chưa thay thế vai trò tư vấn chính thức của luật sư hoặc chuyên gia pháp lý, mà đóng vai trò hỗ trợ tra cứu, tóm tắt và giải thích thông tin từ tài liệu được cung cấp.

- Nguồn tri thức chính là các tài liệu do người dùng tải lên hệ thống.
- Hệ thống hỗ trợ các định dạng tài liệu cơ bản: PDF, TXT và Markdown.
- Quá trình xử lý PDF ưu tiên trích xuất text trực tiếp; OCR chỉ được sử dụng khi cần.
- Câu trả lời được sinh dựa trên các đoạn tài liệu liên quan và có thể bổ sung web fallback khi thiếu thông tin.
- Hệ thống tập trung vào chức năng hỏi đáp, xử lý tài liệu, quản lý người dùng và lưu lịch sử hội thoại.

1.4. Cơ sở lý thuyết

1.4.1. Chatbot pháp luật

Chatbot pháp luật là hệ thống hỗ trợ người dùng đặt câu hỏi liên quan đến quy định, điều khoản, thủ tục hoặc nội dung trong văn bản pháp luật. Người dùng có thể nhập câu hỏi bằng ngôn ngữ tự nhiên, hệ thống sẽ xử lý, tìm kiếm thông tin liên quan và trả về câu trả lời dễ hiểu.

Một chatbot pháp luật cần có khả năng trả lời dựa trên căn cứ, hạn chế suy diễn ngoài tài liệu, giải thích nội dung phức tạp theo cách rõ ràng và có thể truy xuất đúng đoạn văn

bản liên quan. Đây là điểm khác biệt quan trọng giữa chatbot pháp luật và chatbot trò chuyện thông thường.

1.4.2. Retrieval-Augmented Generation

Retrieval-Augmented Generation, viết tắt là RAG, là phương pháp kết hợp giữa truy xuất thông tin và sinh ngôn ngữ tự nhiên. Khi người dùng đặt câu hỏi, hệ thống không yêu cầu mô hình ngôn ngữ lớn trả lời ngay, mà trước tiên tìm kiếm các đoạn tài liệu liên quan trong kho dữ liệu. Các đoạn này sau đó được đưa vào prompt làm ngữ cảnh để mô hình sinh câu trả lời.

Phương pháp RAG phù hợp với lĩnh vực pháp luật vì câu trả lời cần bám sát nguồn tài liệu cụ thể. Nếu kho dữ liệu được xây dựng tốt, RAG giúp giảm hiện tượng hallucination và tăng khả năng kiểm chứng của câu trả lời.

1.4.3. Vector embedding và Hybrid Search

Vector embedding là quá trình biểu diễn văn bản dưới dạng vector số học trong không gian nhiều chiều. Các đoạn văn bản gần nhau về mặt ngữ nghĩa sẽ có vector gần nhau. Nhờ vậy, hệ thống có thể tìm kiếm theo ý nghĩa của câu hỏi thay vì chỉ dựa trên từ khóa trùng khớp.

Hybrid Search là kỹ thuật kết hợp giữa tìm kiếm ngữ nghĩa và tìm kiếm từ khóa. Trong văn bản pháp luật, kỹ thuật này rất quan trọng vì nhiều truy vấn cần khớp chính xác với tên văn bản, số điều, số khoản, mốc thời gian hoặc thuật ngữ pháp lý, đồng thời vẫn cần hiểu được ý nghĩa của câu hỏi.

1.4.4. Rerank

Rerank là bước sắp xếp lại các kết quả truy xuất. Sau khi hệ thống tìm được nhiều đoạn văn bản ứng viên, mô hình rerank đánh giá lại mức độ liên quan giữa câu hỏi và từng đoạn. Các đoạn phù hợp nhất sẽ được chọn làm ngữ cảnh cho mô hình sinh câu trả lời. Bước này giúp giảm nhiễu và cải thiện chất lượng đầu ra.

PHẦN 2. THIẾT KẾ

2.1. Yêu cầu hệ thống

2.1.1. Yêu cầu chức năng

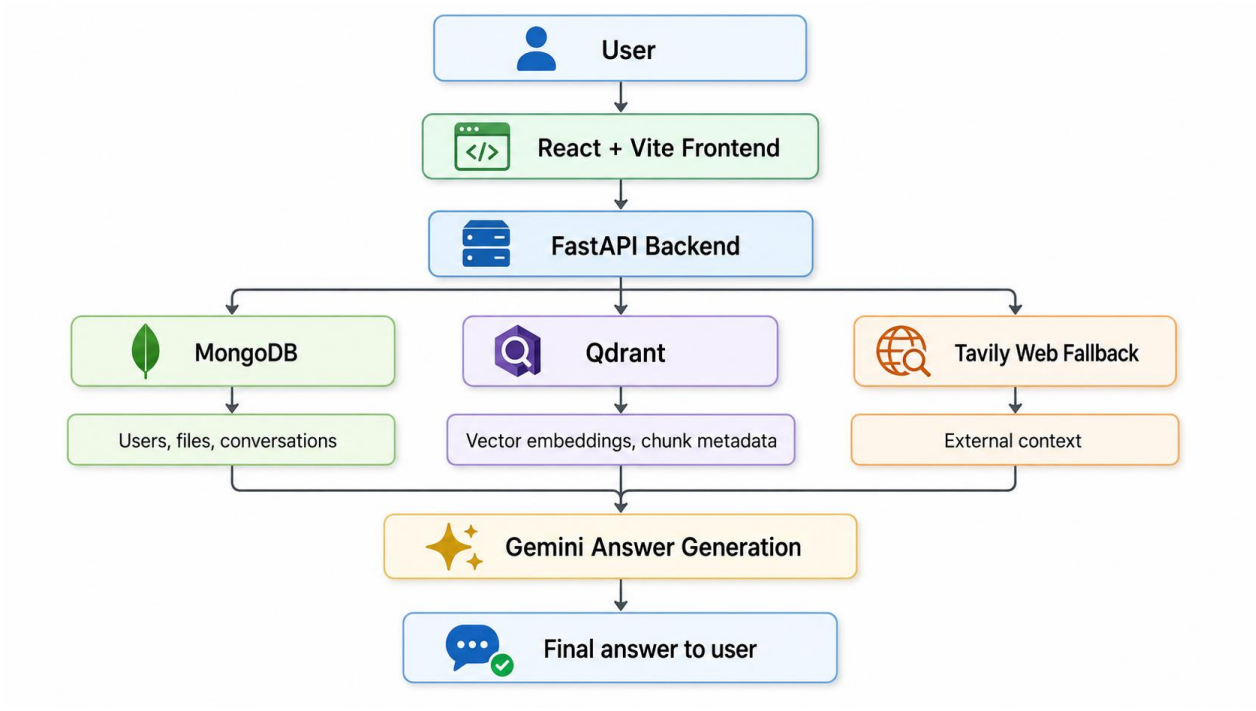
Chức năng	Mô tả
Đăng ký, đăng nhập	Người dùng có thể tạo tài khoản, đăng nhập và sử dụng hệ thống theo tài khoản riêng.
Bảo mật mật khẩu	Mật khẩu được mã hóa bằng bcrypt trước khi lưu vào MongoDB.
Tải tài liệu	Người dùng có thể tải lên PDF, TXT hoặc Markdown để làm nguồn tri thức.
Xử lý tài liệu	Backend trích xuất text, OCR khi cần và chia nhỏ nội dung thành các chunk.
Hỏi đáp chatbot	Người dùng đặt câu hỏi bằng ngôn ngữ tự nhiên và nhận câu trả lời từ hệ thống.
Truy xuất tài liệu	Hệ thống sử dụng Hybrid Search, query decomposition và rerank để chọn ngữ cảnh.
Web fallback	Tavily được sử dụng khi tài liệu nội bộ không đủ thông tin để trả lời.
Lưu lịch sử	Các cuộc trò chuyện được lưu trong MongoDB để người dùng có thể xem lại.

2.1.2. Yêu cầu phi chức năng

- Giao diện dễ sử dụng, phù hợp với người dùng không chuyên kỹ thuật.
- Backend có cấu trúc rõ ràng, dễ bảo trì và mở rộng.
- Dữ liệu người dùng được lưu trữ an toàn, mật khẩu không lưu dưới dạng văn bản gốc.
- Hệ thống có khả năng xử lý tài liệu PDF scan bằng OCR offline.
- Câu trả lời cần bám sát ngữ cảnh truy xuất, hạn chế sinh thông tin không có căn cứ.
- Các thành phần hạ tầng có thể khởi chạy bằng Docker Compose.
- Hệ thống nên ưu tiên Python 3.11 hoặc 3.12 trên Windows để tránh lỗi tương thích thư viện ML.

2.2. Kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình full-stack gồm ba lớp chính: lớp giao diện người dùng, lớp xử lý nghiệp vụ và lớp lưu trữ/hạ tầng. Frontend React + Vite chịu trách nhiệm hiển thị giao diện. Backend FastAPI xử lý API, nghiệp vụ RAG và giao tiếp với các dịch vụ khác. MongoDB và Qdrant đảm nhiệm vai trò lưu trữ dữ liệu ứng dụng và vector embedding.



Kiến trúc này giúp tách biệt rõ trách nhiệm giữa các thành phần. Frontend không xử lý logic AI phức tạp mà chỉ gửi yêu cầu và hiển thị kết quả. Backend đóng vai trò điều phối toàn bộ pipeline. Qdrant chuyên lưu trữ vector, còn MongoDB lưu dữ liệu nghiệp vụ như người dùng, tài liệu và hội thoại.

2.3. Thiết kế dữ liệu

Dữ liệu của hệ thống được chia thành hai nhóm chính: dữ liệu ứng dụng và dữ liệu vector. Dữ liệu ứng dụng được lưu trong MongoDB, còn dữ liệu vector được lưu trong Qdrant. Cách tách này giúp việc quản lý rõ ràng và phù hợp với đặc điểm từng loại dữ liệu.

Thành phần	Dữ liệu lưu trữ	Vai trò
MongoDB - users	Thông tin tài khoản, mật khẩu đã hash	Xác thực và quản lý người dùng
MongoDB - documents	Tên file, đường dẫn, metadata, người tải lên	Quản lý tài liệu
MongoDB - conversations	Câu hỏi, câu trả lời, thời gian, session	Lưu lịch sử hội thoại
Qdrant - vectors	Embedding của chunk và payload metadata	Truy xuất ngữ nghĩa
Local storage	File gốc trong backend/storage/uploads	Lưu file tải lên

2.3.1. Metadata của chunk

Mỗi chunk không chỉ lưu nội dung văn bản mà còn cần metadata để phục vụ truy xuất và hiển thị nguồn. Metadata có thể bao gồm tên tài liệu, số trang, tiêu đề chương, tiêu đề mục, điều khoản, ID người dùng và thời điểm tải lên. Với văn bản pháp luật, metadata giúp hệ thống biết đoạn văn thuộc phần nào trong tài liệu, từ đó cải thiện khả năng trả lời có căn cứ.

2.4. Thiết kế luồng xử lý RAG

Pipeline RAG của hệ thống được thiết kế thành hai giai đoạn chính: giai đoạn nạp tài liệu và giai đoạn hỏi đáp. Giai đoạn nạp tài liệu xử lý file đầu vào, tạo chunk, sinh embedding và lưu vào Qdrant. Giai đoạn hỏi đáp xử lý câu hỏi, truy xuất ngữ cảnh, rerank và gọi Gemini để sinh câu trả lời.

2.4.1. Luồng nạp tài liệu

- Người dùng tải lên tài liệu PDF, TXT hoặc Markdown.
- Backend lưu file vào thư mục backend/storage/uploads.
- Hệ thống trích xuất text trực tiếp nếu file có sẵn nội dung văn bản.
- Nếu PDF không có text hoặc text quá ít, hệ thống dùng OCR offline bằng PyMuPDF và RapidOCR.
- Nội dung tài liệu được chuẩn hóa và chia nhỏ theo header.
- Mỗi chunk được mã hóa thành embedding.
- Embedding và metadata được lưu vào Qdrant; thông tin tài liệu được lưu vào MongoDB.

2.4.2. Luồng hỏi đáp

- Người dùng nhập câu hỏi trên giao diện chatbot.
- Backend nhận câu hỏi và phân rã thành các truy vấn con nếu cần.
- Hệ thống thực hiện Hybrid Search trên kho tài liệu nội bộ.
- Các kết quả ứng viên được đưa qua bước rerank để chọn ngữ cảnh tốt nhất.
- Nếu ngữ cảnh nội bộ chưa đủ mạnh, Tavily được sử dụng để bổ sung thông tin web.
- Gemini nhận câu hỏi và ngữ cảnh, sau đó sinh câu trả lời cuối cùng.
- Câu trả lời được hiển thị trên frontend và lưu vào lịch sử hội thoại.

2.5. Thiết kế giao diện và API

Frontend được thiết kế xoay quanh các màn hình chính: đăng nhập/đăng ký, tải tài liệu, giao diện chatbot và lịch sử hội thoại. Người dùng có thể đăng nhập, tải tài liệu và đặt câu

hỏi trong cùng một hệ thống. Giao diện cần đơn giản để người dùng tập trung vào việc hỏi đáp và xem kết quả.

Backend có thể tổ chức thành các nhóm API: API xác thực, API tài liệu, API chatbot và API lịch sử. Cách chia này giúp mã nguồn rõ ràng, mỗi nhóm API chịu trách nhiệm cho một nhóm nghiệp vụ cụ thể.

Nhóm API	**Chức năng chính**
Auth API	Đăng ký, đăng nhập, kiểm tra thông tin người dùng
Document API	Tải tài liệu, xử lý tài liệu, lấy danh sách tài liệu
Chat API	Nhận câu hỏi, truy xuất ngữ cảnh, sinh câu trả lời
History API	Lấy và quản lý lịch sử hội thoại

PHẦN 3. CÀI ĐẶT

3.1. Môi trường và công nghệ sử dụng

Dự án được xây dựng với các công nghệ chính gồm FastAPI cho backend, React + Vite cho frontend, MongoDB để lưu dữ liệu ứng dụng, Qdrant để lưu vector, PyMuPDF và RapidOCR để xử lý PDF/OCR, Tavily làm web fallback và Gemini để sinh câu trả lời.

Công nghệ	Vai trò trong hệ thống
FastAPI	Xây dựng backend API, xử lý nghiệp vụ và điều phối pipeline RAG
React + Vite	Xây dựng giao diện người dùng và giao tiếp với backend
PyMuPDF	Trích xuất text từ PDF có sẵn nội dung văn bản
RapidOCR	Nhận dạng văn bản trong PDF scan bằng OCR offline
Qdrant	Lưu trữ vector embedding và metadata phục vụ truy xuất ngữ nghĩa
MongoDB	Lưu tài khoản, tài liệu tải lên và lịch sử hội thoại
Tavily	Tìm kiếm web dự phòng khi tài liệu nội bộ chưa đủ thông tin
Gemini	Sinh câu trả lời cuối cùng dựa trên ngữ cảnh được truy xuất
Docker Compose	Khởi chạy các dịch vụ hạ tầng như MongoDB và Qdrant

3.2. Cấu trúc thư mục dự án

Cấu trúc thư mục được tổ chức rõ ràng, tách riêng backend, frontend và hạ tầng. Điều này giúp quá trình phát triển, kiểm thử và bảo trì thuận tiện hơn.

```
.
|-- backend
|   |-- app
|   |   |-- api
|   |   |-- core
|   |   |-- db
|   |   |-- models
|   |   |-- schemas
|   |   |-- services
|   |   `-- utils
|   |-- requirements.txt
|   `-- storage
|-- frontend
|   |-- public
|   |-- src
|   `-- package.json
|-- docker-compose.yml
`-- README.md
```

Trong backend, thư mục app/api chứa các route API, app/core chứa cấu hình hệ thống, app/db quản lý kết nối cơ sở dữ liệu, app/models và app/schemas định nghĩa dữ liệu,

app/services chứa logic nghiệp vụ và app/utils chứa các hàm tiện ích. Thư mục storage dùng để lưu các file người dùng tải lên.

Trong frontend, thư mục src chứa mã nguồn React, public chứa tài nguyên tĩnh và package.json khai báo dependencies cũng như script chạy ứng dụng.

3.3. Cài đặt hạ tầng

Trước khi chạy backend và frontend, cần sao chép các file môi trường và khởi động các dịch vụ hạ tầng. File .env được tạo từ .env.example và cần điền các khóa bí mật như GEMINI_API_KEY, TAVILY_API_KEY cùng cấu hình kết nối MongoDB, Qdrant.

```
cd backend
cp .env.example .env

cd ../frontend
cp .env.example .env
```

Sau khi cấu hình môi trường, khởi động hạ tầng bằng Docker Compose:

```
docker compose up -d
```

Lệnh trên giúp khởi chạy các dịch vụ cần thiết cho hệ thống, đặc biệt là MongoDB và Qdrant. Các dịch vụ này cần chạy trước khi backend thực hiện lưu dữ liệu hoặc truy vấn vector.

3.4. Cài đặt backend

Backend được chạy trong môi trường ảo Python để tránh xung đột thư viện. Trên Windows, nên sử dụng Python 3.11 hoặc 3.12 vì một số gói machine learning có thể chưa ổn định với Python 3.14.

```
cd backend
py -3.11 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Sau khi chạy thành công, backend thường hoạt động tại địa chỉ http://127.0.0.1:8000. FastAPI cũng có thể cung cấp tài liệu API tự động để hỗ trợ kiểm thử các endpoint trong quá trình phát triển.

3.4.1. Cài đặt xác thực người dùng

Chức năng xác thực được triển khai bằng cách lưu thông tin người dùng trong MongoDB. Khi đăng ký, mật khẩu được hash bằng bcrypt trước khi lưu. Khi đăng nhập, hệ thống so sánh mật khẩu người dùng nhập với bản hash đã lưu để xác thực.

3.4.2. Cài đặt xử lý tài liệu

Backend nhận file từ frontend, lưu vào backend/storage/uploads, sau đó xác định loại file để xử lý. Với TXT và Markdown, hệ thống có thể đọc trực tiếp nội dung. Với PDF, hệ thống dùng PyMuPDF để trích xuất text trước. Nếu PDF không có text đủ dùng, RapidOCR sẽ được gọi để nhận dạng nội dung từ ảnh.

3.4.3. Cài đặt lưu trữ Qdrant và MongoDB

Sau khi tài liệu được chia thành chunk và sinh embedding, các vector được lưu vào Qdrant kèm metadata. Thông tin tài liệu, người dùng và lịch sử hội thoại được lưu vào MongoDB. Cách tách lưu trữ này giúp hệ thống dễ mở rộng khi số lượng tài liệu tăng.

3.5. Cài đặt frontend

Frontend được cài đặt bằng Node.js và npm. Sau khi cài dependencies, có thể chạy ứng dụng ở chế độ phát triển bằng Vite.

```
cd frontend
npm install
npm run dev
```

Sau khi chạy thành công, giao diện thường được mở tại http://localhost:5173/. Người dùng có thể truy cập trang này để đăng nhập, tải tài liệu và sử dụng chatbot.

Frontend gửi yêu cầu đến backend thông qua các API. Khi người dùng tải tài liệu, frontend gửi file lên backend. Khi người dùng đặt câu hỏi, frontend gửi nội dung câu hỏi và nhận lại câu trả lời từ backend để hiển thị trong giao diện hội thoại.

3.6. Cài đặt các thành phần RAG

3.6.1. OCR offline

OCR offline được triển khai nhằm xử lý các file PDF dạng scan. Nếu PDF đã có text, hệ thống ưu tiên sử dụng text có sẵn để tiết kiệm thời gian và tăng độ chính xác. Nếu không có text, PyMuPDF hỗ trợ chuyển trang PDF thành ảnh và RapidOCR nhận dạng nội dung chữ trên ảnh.

3.6.2. Header-based chunking

Chiến lược chunking theo header được sử dụng để giữ lại cấu trúc tài liệu pháp luật. Thay vì cắt văn bản đơn thuần theo số ký tự, hệ thống cố gắng chia theo chương, mục, điều hoặc tiêu đề phụ. Điều này giúp mỗi chunk có ngữ cảnh rõ ràng hơn.

3.6.3. Query decomposition

Query decomposition giúp phân rã câu hỏi phức tạp thành nhiều truy vấn con. Ví dụ, nếu người dùng hỏi so sánh hai quy định hoặc hỏi nhiều ý trong một câu, hệ thống có thể tạo ra các sub-query để truy xuất thông tin đầy đủ hơn.

3.6.4. Hybrid Search và rerank

Hybrid Search kết hợp truy xuất vector và truy xuất từ khóa. Sau khi có danh sách kết quả ứng viên, bước rerank được sử dụng để đánh giá lại mức độ liên quan giữa câu hỏi và từng chunk. Các chunk có điểm cao nhất được chọn làm ngữ cảnh cho Gemini.

3.6.5. Tavily web fallback

Tavily web fallback được sử dụng khi dữ liệu nội bộ không đủ để trả lời câu hỏi. Thành phần này giúp hệ thống bổ sung thông tin từ bên ngoài, tuy nhiên vẫn cần ưu tiên nguồn tài liệu nội bộ để đảm bảo câu trả lời bám sát dữ liệu đã tải lên.

3.6.6. Gemini answer generation

Gemini nhận câu hỏi của người dùng cùng với ngữ cảnh được truy xuất. Prompt cần được thiết kế để mô hình trả lời bằng tiếng Việt, bám sát ngữ cảnh, không bịa thêm điều luật và thông báo khi không đủ thông tin để kết luận.

PHẦN 4. KIỂM THỬ

4.1. Mục tiêu kiểm thử

Mục tiêu của kiểm thử là đánh giá xem hệ thống có đáp ứng các chức năng đã thiết kế hay không. Các nội dung kiểm thử tập trung vào đăng nhập, tải tài liệu, trích xuất nội dung, OCR, chunking, truy xuất, sinh câu trả lời và lưu lịch sử hội thoại.

- Kiểm tra người dùng có thể đăng ký, đăng nhập và sử dụng hệ thống.
- Kiểm tra file PDF, TXT và Markdown có thể được tải lên và xử lý.
- Kiểm tra PDF có sẵn text được trích xuất trực tiếp bằng PyMuPDF.
- Kiểm tra PDF scan được xử lý bằng OCR offline khi cần.
- Kiểm tra nội dung được chia chunk theo header hợp lý.
- Kiểm tra truy xuất Hybrid Search trả về các đoạn liên quan.
- Kiểm tra rerank đưa các đoạn phù hợp nhất lên đầu.
- Kiểm tra Tavily được gọi khi context nội bộ không đủ mạnh.
- Kiểm tra Gemini sinh câu trả lời rõ ràng, bám sát ngữ cảnh.
- Kiểm tra lịch sử hội thoại được lưu và có thể xem lại.

4.2. Kịch bản kiểm thử

Mã kiểm thử	Chức năng	Dữ liệu đầu vào	Kết quả mong đợi
TC01	Đăng ký tài khoản	Thông tin người dùng hợp lệ	Tài khoản được tạo, mật khẩu được hash trong MongoDB
TC02	Đăng nhập	Email và mật khẩu đúng	Người dùng đăng nhập thành công và vào giao diện chính
TC03	Tải file TXT	Một file văn bản pháp luật dạng TXT	Nội dung được đọc và xử lý thành công
TC04	Tải file Markdown	Một file Markdown có tiêu đề	Nội dung được đọc và chia theo cấu trúc tiêu đề
TC05	Tải PDF có text	PDF chứa text có thể chọn được	Hệ thống trích xuất text bằng PyMuPDF, không cần OCR
TC06	Tải PDF scan	PDF dạng ảnh hoặc không có text	Hệ thống fallback sang OCR offline bằng RapidOCR
TC07	Chunking theo header	Tài liệu có chương, mục, điều	Chunk giữ được thông tin header và ngữ cảnh
TC08	Hỏi đáp nội bộ	Câu hỏi có thông tin trong tài liệu	Trả lời dựa trên chunk liên quan

Mã kiểm thử	Chức năng	Dữ liệu đầu vào	Kết quả mong đợi
TC09	Hỏi đáp thiếu dữ liệu	Câu hỏi không đủ thông tin nội bộ	Hệ thống sử dụng Tavily hoặc thông báo thiếu căn cứ
TC10	Lưu lịch sử	Một phiên hỏi đáp hoàn chỉnh	Câu hỏi và câu trả lời được lưu trong MongoDB

4.3. Đánh giá kết quả

Qua các kịch bản kiểm thử, hệ thống đáp ứng được các chức năng cốt lõi của một chatbot RAG pháp luật. Người dùng có thể đăng nhập, tải tài liệu, đặt câu hỏi và nhận câu trả lời dựa trên nội dung tài liệu. Việc kết hợp PyMuPDF và RapidOCR giúp hệ thống xử lý cả PDF có text và PDF scan.

Chiến lược header-based chunking giúp hệ thống giữ lại cấu trúc của văn bản pháp luật, từ đó cải thiện khả năng truy xuất đúng điều khoản hoặc phần nội dung liên quan. Hybrid Search giúp cân bằng giữa tìm kiếm theo ngữ nghĩa và tìm kiếm theo từ khóa chính xác. Bước rerank tiếp tục giúp giảm nhiễu trong kết quả truy xuất.

Về mặt sinh câu trả lời, Gemini có khả năng tạo câu trả lời tự nhiên, rõ ràng và dễ hiểu khi được cung cấp ngữ cảnh phù hợp. Tuy nhiên, chất lượng câu trả lời vẫn phụ thuộc vào chất lượng tài liệu đầu vào, độ chính xác của OCR và chất lượng truy xuất.

4.4. Một số trường hợp kiểm thử minh họa

Tình huống	Cách hệ thống xử lý	Nhận xét
Người dùng hỏi đúng nội dung có trong tài liệu	Hybrid Search tìm chunk liên quan, rerank chọn đoạn tốt nhất, Gemini trả lời dựa trên ngữ cảnh	Kết quả phù hợp với mục tiêu RAG
Người dùng hỏi câu dài gồm nhiều ý	Query decomposition tách thành các truy vấn con để truy xuất nhiều nhóm thông tin	Tăng khả năng bao phủ thông tin
Tài liệu PDF không có text	Hệ thống fallback sang OCR offline	Giúp xử lý tài liệu scan trong thực tế
Tài liệu nội bộ thiếu thông tin	Hệ thống có thể gọi Tavily web fallback hoặc thông báo chưa đủ căn cứ	Giảm rủi ro trả lời bịa
Người dùng hỏi tiếp trong cùng phiên	Lịch sử hội thoại được dùng để duy trì ngữ cảnh khi cần	Tăng tính tự nhiên của trải nghiệm

PHẦN 5. KẾT LUẬN

5.1. Kết quả đạt được

Đề tài "Xây dựng hệ thống Chatbot pháp luật ứng dụng RAG" đã xây dựng được một ứng dụng full-stack có khả năng hỗ trợ người dùng hỏi đáp dựa trên tài liệu pháp luật được tải lên. Hệ thống kết hợp nhiều thành phần quan trọng như xử lý tài liệu, OCR offline, chunking theo header, embedding, Qdrant, MongoDB, Hybrid Search, rerank, Tavily web fallback và Gemini answer generation.

- Xây dựng backend FastAPI có khả năng xử lý API, tài liệu, truy xuất và sinh câu trả lời.
- Xây dựng frontend React + Vite để người dùng đăng nhập, tải tài liệu và tương tác với chatbot.
- Tích hợp xử lý PDF bằng PyMuPDF và OCR offline bằng RapidOCR.
- Áp dụng chiến lược header-based chunking để giữ cấu trúc văn bản pháp luật.
- Lưu vector embedding trong Qdrant và dữ liệu ứng dụng trong MongoDB.
- Triển khai pipeline hỏi đáp gồm query decomposition, Hybrid Search, rerank và Gemini.
- Bổ sung Tavily web fallback để hỗ trợ khi dữ liệu nội bộ chưa đủ thông tin.
- Lưu lịch sử hội thoại, giúp người dùng có thể theo dõi lại quá trình hỏi đáp.

5.2. Hạn chế

Mặc dù đã hoàn thiện các chức năng cốt lõi, hệ thống vẫn còn một số hạn chế cần tiếp tục cải thiện trong các phiên bản sau.

- Chất lượng OCR có thể giảm nếu tài liệu scan bị mờ, nghiêng, nhiều bảng biểu hoặc bố cục phức tạp.
- Chunking theo header phụ thuộc vào khả năng nhận diện đúng cấu trúc tài liệu.
- Chất lượng câu trả lời phụ thuộc vào chất lượng tài liệu đầu vào và kết quả truy xuất.
- Hệ thống chưa có cơ chế trích dẫn nguồn chi tiết theo từng câu trả lời.
- Chưa có cơ chế đánh giá tự động độ chính xác của câu trả lời.
- Chưa triển khai phân quyền nâng cao cho nhiều vai trò người dùng.
- Tavily web fallback cần được kiểm soát chặt để tránh đưa nguồn chưa phù hợp vào câu trả lời pháp luật.

5.3. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng từ một chatbot hỏi đáp thành một trợ lý pháp luật thông minh với nhiều chức năng hơn. Một số hướng phát triển bao gồm:

- Bổ sung trích dẫn nguồn rõ ràng theo từng đoạn trả lời.
- Hỗ trợ thêm các định dạng tài liệu như DOCX, XLSX hoặc HTML.
- Cải thiện OCR cho tài liệu nhiều cột, bảng biểu, dấu mộc hoặc chữ nhỏ.
- Xây dựng dashboard quản lý tài liệu, thống kê số lượt hỏi và chất lượng truy xuất.
- Bổ sung cơ chế phân quyền người dùng như admin, người dùng thường và chuyên viên pháp lý.
- Hỗ trợ tóm tắt văn bản pháp luật tự động.
- Hỗ trợ so sánh nhiều văn bản pháp luật để tìm điểm khác nhau.
- Hỗ trợ phát hiện điều khoản mâu thuẫn, đã hết hiệu lực hoặc cần kiểm tra nguồn mới.
- Tối ưu prompt và cơ chế kiểm soát hallucination cho lĩnh vực pháp luật.
- Triển khai đánh giá định lượng cho retrieval, rerank và chất lượng câu trả lời.

5.4. Kết luận chung

Thông qua quá trình thực hiện bài tập lớn, nhóm đã tiếp cận và vận dụng nhiều công nghệ hiện đại trong lĩnh vực trí tuệ nhân tạo và phát triển ứng dụng web. Hệ thống Chatbot pháp luật ứng dụng RAG cho thấy tiềm năng lớn trong việc hỗ trợ người dùng tra cứu, hiểu và khai thác văn bản pháp luật hiệu quả hơn.

So với chatbot truyền thống, hệ thống RAG có ưu điểm là câu trả lời được sinh dựa trên tài liệu cụ thể, từ đó giảm nguy cơ sai lệch thông tin. Việc bổ sung OCR offline giúp hệ thống xử lý được nhiều dạng tài liệu thực tế hơn, đặc biệt là các file PDF scan thường gặp trong môi trường hành chính và pháp lý.

Đề tài là nền tảng ban đầu để tiếp tục phát triển thành một trợ lý pháp luật thông minh, không chỉ hỏi đáp mà còn có thể tóm tắt, phân tích, so sánh văn bản, hỗ trợ tra cứu thủ tục và đưa ra gợi ý dựa trên căn cứ pháp lý rõ ràng.

TÀI LIỆU THAM KHẢO

- README của dự án Chatbot RAG Project.
- Tài liệu kỹ thuật FastAPI.
- Tài liệu kỹ thuật React và Vite.
- Tài liệu kỹ thuật MongoDB.
- Tài liệu kỹ thuật Qdrant.

- Tài liệu sử dụng PyMuPDF và RapidOCR.
- Tài liệu về Retrieval-Augmented Generation và các hệ thống hỏi đáp dựa trên truy xuất.